Var vs Let vs Const | Hoisting | Data Types in JavaScript | typeOf Method

Var vs Let vs Const:

Feature / Keyword	var	let	const
Scope	Function-scoped	Block-scoped ({})	Block-scoped ({})
Hoisting	Hoisted to top of scope, initialized as undefined	Hoisted but error	Hoisted but error
Re-declaration	Allowed within the same scope	Not allowed within the same scope	Not allowed within the same scope
Re-assignment	Allowed	Allowed	**Not allowed** (constant reference)
Initialization Required	No	No	**Yes** — must be initialized when declared
Use in Loops	Can cause issues with closures in loops	Safer in loops, creates a new binding per iteration	Same behavior as let

Feature	Code	output

Re-declaration	// var
	var x = 10;
	var x = 20; // ✅ Allowed
	
	// let
	let y = 30;
	// let y = 40; ❌ SyntaxError: Identifier 'y' has already been declared
	
	// const
	const z = 50;
	// z = 60; ❌ TypeError: Assignment to constant variable	

Scope	function scopeTest() {
	if (true) {
	var varVariable = "I am var";
	let letVariable = "I am let";
	const constVariable = "I am const";
	
	console.log("Inside if block:");
	console.log(varVariable); // ✅ Accessible
	console.log(letVariable); // ✅ Accessible
	console.log(constVariable); // ✅ Accessible
	}
	
	console.log("Outside if block:");
	console.log(varVariable); // ✅ Accessible — function scoped
	// console.log(letVariable); // ❌ ReferenceError: letVariable is not defined
	// console.log(constVariable); // ❌ ReferenceError: constVariable is not defined
	}
	
	scopeTest(); //calling the function
		Inside if block:
		I am var
		I am let
		I am const
		Outside if block:
		I am var
		Uncaught ReferenceError: letVariable is not defined
		Uncaught ReferenceError: constVariable is not defined

Scope	//Var: function scope
	function test() {
	var a = 20; //a is declared, and can be used anywhere in function
	console.log(a);
	if (true) {
	a = 30;
	console.log(a);
	}
	console.log(a);
	}
	
	test();//calling test function
		20
		30
		30

Scope	//let: block scope
	function rest() {
	if (true) {
	let b = 40; //b is declared here
	console.log(b);
	}
	console.log(b);
	}
	
	rest();//calling the function
		40
		ReferenceError: b is not defined

Scope	var a = 10; //global variable
	function test() {
	console.log(a);//undefined
	var a = 20; //declared here
	console.log(a);//20
	if (true) {
	var a = 30;
	console.log(a);//30
	}
	console.log(a);//30
	}
	test();
	console.log(a);//10
		undefined
		20
		30
		30
		10

Scope	function nest() {
	var r = 10;
	if (true) {
	var r = 20;
	console.log(r);
	}
	if (10 > 3) {
	console.log(r);
	}
	console.log(r);
	}
	nest();
		20
		20
		20

Scope	function nest() {
	var r = 10;
	if (true) {
	let r = 20;
	console.log(r);
	}
	if (10 > 3) {
	console.log(r);
	}
	console.log(r);
	}
	nest();
		20
		10
		10

Scope	function nest() {
	let r = 10;
	if (true) {
	let r = 20;
	console.log(r);
	}
	if (10 > 3) {
	let r = 30;
	console.log(r);
	}
	console.log(r);
	}
	nest();
		20
		30
		10

Keyword	Scope Type	Accessible outside block {}?
var	Function scope	✅ Yes
let	Block scope	❌ No
const	Block scope	❌ No

What is Hoisting?

Hoisting is JavaScript's default behavior of **moving declarations to the top** of their scope (before code execution).

var Hoisting Example

```
console.log(a); // 👉 undefined (NOT ReferenceError)
var a = 10;
```

let and const Hoisting Example

```
console.log(b); // ❌ ReferenceError: Cannot access 'b' before initialization
let b = 20;
```

```
console.log(c); // ❌ ReferenceError: Cannot access 'c' before initialization
const c = 30;
```

Feature	var	let / const
Declaration Hoisted	✅ Yes	✅ Yes
Access Before Declaration	✅ Allowed (undefined)	❌ ReferenceError

const — Fixed (Constant) Values

- A **const variable must be assigned a value at the time of declaration**.
- Once assigned, **you cannot reassign it** — the **reference is constant**.

```
const PI = 3.14;
PI = 3.14159;  // ❌ TypeError: Assignment to constant variable.

//Other Examples:
const DAYS_IN_WEEK = 7;
const APP_NAME = "OpenApp";
const API_BASE_URL = "https://naveenautomationlabs.com";
```

Important Points:

- var: can be redeclared
  - var a = 10;
  - var = 10; //No Error
- var: can be reassigned
  - var t = 10;
  - t = 20;
  - clg(t); //will print 20
- let: can not be redeclared
  - let b = 10;
  - let b = 20; //Error
- let: can be reassigned
  - let p = 10;
  - p = 20;
  - clg(p); //will print 20
- const: should be initialized with some value
  - const days = 7;
  - const total; //Error — total should be initialized with some initial value
  - const, can not be reassigned. It's a fixed value
  - const, can not be redeclared.

DataTypes in Javascript:

NonPrimitive[Reference Type]:

Object
Array
Function

Primitive:

Number
BigInt
String
Boolean
undefined
null

Data Type	Example	Description	Approx. Size
Number	42, 3.14, -0.5	64-bit floating point (IEEE 754)	8 bytes (64 bits)
BigInt	12345678901234567890n	Arbitrary precision integer	Depends on digits
String	"hello", 'abc'	Sequence of UTF-16 code units	2 bytes per character

Boolean	true, false	Logical values	1 byte (typically)
undefined	undefined	Default uninitialized value	0 to 8 bytes (engine-dependent)
null	null	Explicitly empty value	0 to 8 bytes (engine-dependent)
Symbol	Symbol("id")	Unique and immutable primitive identifier	Cannot be measured directly
Object	{a: 1}, new Object()	Collection of key-value pairs	Varies (depends on props, nesting, etc.)
Array	[1, 2, 3]	Ordered list	Depends on elements
Function	function() {}	Callable object	Varies (depends on closure)

```
const str = "hello"; // 5 characters × 2 bytes = 10 bytes approx.
const num = 100;     // 8 bytes
const flag = true;   // 4 bytes
```

What is typeOf?

-is used to get the data type of a value/variable

Value / Expression	Type	typeof Result
42	Number	"number"
3.14	Number	"number"
NaN	Not-a-Number (still number)	"number"
Infinity	Number (special)	"number"
"hello"	String	"string"
`template`	String (Template)	"string"
true / false	Boolean	"boolean"
undefined	Undefined	"undefined"
null	Null	"object" ! (quirk) - JS legacy bug
Symbol("id")	Symbol	"symbol"
BigInt(1234567890123)	BigInt	"bigint"
[]	Array (object type)	"object"
{}	Object	"object"

function() {}	Function	"function"
() => {}	Arrow Function	"function"
class A {}	Class (special function)	"function"
new Date()	Date object	"object"
/abc/	RegExp object	"object"
new Map()	Map	"object"
new Set()	Set	"object"
Math	Built-in object	"object"
JSON	Built-in object	"object"

Note: More data types and values will be covered in upcoming chapters.